

# Smile Detection using Different Classification approaches

Sarah Camila Guzmán Fierro<sup>1</sup>, Gonzalo Makenly Higuera Inzunza<sup>1</sup> and Juan Pablo de la Peña González<sup>1</sup>

<sup>1</sup> *Instituto Tecnológico de Monterrey*

Reception date of the manuscript: 03/12/2025

Acceptance date of the manuscript: –

Publication date: –

**Abstract**— Facial expression analysis has become a relevant field in computer vision, particularly for tasks such as emotion recognition, human–computer interaction, and behavioral analytics. In this study, we explore the problem of smile detection using three machine–learning approaches with distinct conceptual foundations. A supervised CNN was trained to classify images directly, an Autoencoder was adapted to detect smiles as deviations from a baseline reconstruction error, and a Random Forest was trained on vectorized image pixels. All methods were trained and evaluated using the same dataset. Evaluation metrics suggest meaningful differences in generalization capacity, computation time, and robustness across models.

**Keywords**—Smile Detection, Convolutional Neural Networks, Autoencoder, Random Forest, Facial Expression Recognition, Computer Vision, Machine Learning, Anomaly Detection

## I. SMILE DETECTION EXPERIMENT

### a. Overview

This section presents an experimental study focused on the automatic detection of smiles using image–based machine learning models. Three different approaches were implemented and evaluated under comparable conditions: a Convolutional Neural Network (CNN), an Autoencoder used for anomaly detection, and a classical Random Forest classifier. The purpose of this study is to determine which model offers the best balance between predictive performance, computational efficiency, and ease of deployment in real–world applications.

## II. INTRODUCTION

Smile detection can be framed as a binary classification problem where the goal is to determine whether a face image contains a smiling expression or not. Beyond its academic interest, this task has growing relevance in numerous real–world applications such as affective computing, workplace behavior monitoring, authentication systems enriched with emotional cues, adaptive learning platforms, and personalized user interfaces.

This project implements three distinct approaches that represent different paradigms in machine learning and computer vision: deep learning, classical anomaly detection, and ensemble–based classification. The models included in

the comparison are a Convolutional Neural Network (CNN) trained using supervised learning, an Autoencoder used as an unsupervised detector of anomalies based on reconstruction error thresholds, and Random Forest applied to flattened images as a traditional machine–learning baseline.

Using these three approaches allows examination not only their performance in detecting smiles, but also their computational cost, scalability, and sensitivity to the quality of input data.



**Fig. 1:** Face Comparison

## III. METHODS

### a. Dataset and Preprocessing

The experimental data were loaded and processed using a structured Python pipeline implementing a reproducible preprocessing workflow. The dataset comprised facial images archived in compressed format with corresponding label files for binary classification into *smile* and *non-smile* categories.

## Archive Extraction and File Parsing

The implementation initialized with extraction of facial images from the ZIP archive. Subsequent parsing of label files `SMILE_list.txt` and `NON-SMILE_list.txt` yielded filenames for both classes, with systematic filename modifications to conform to the `.ppm` image format employed throughout the dataset.

## Code Implementation

**Listing 1** Data loading and preprocessing function implementation.

```
def load_data(n=600, target_size=(200,200), test_split=0.2):
    # Extract archive
    with zipfile.ZipFile("data/faces.zip", 'r') as zip_ref:
        zip_ref.extractall("data/faces")

    # Read label files
    path_smile = "data/labels/SMILE_list.txt"
    path_nonSmile = "data/labels/NON-SMILE_list.txt"

    files_smile, files_nonSmile = [], []
    try:
        with open(path_smile, 'r') as f:
            files_smile = [line.strip()[1:-4] + ".ppm" for line in f]
    except FileNotFoundError:
        print(f"Error: File '{path_smile}' not found.")

    try:
        with open(path_nonSmile, 'r') as f:
            files_nonSmile = [line.strip()[1:-4] + ".ppm" for line in f]
    except FileNotFoundError:
        print(f"Error: File '{path_nonSmile}' not found.")

    # Load and preprocess images
    rootpath = "data/faces/"
    imgs_smile, imgs_nonSmile = [], []

    for i in range(n):
        # Smile images
        img_smile = image.load_img(rootpath + files_smile[i],
                                   target_size=target_size)
        imgs_smile.append(image.img_to_array(img_smile) / 255.0)

        # Non-smile images
        img_non = image.load_img(rootpath + files_nonSmile[i],
                                 target_size=target_size)
        imgs_nonSmile.append(image.img_to_array(img_non) / 255.0)

    # Convert to arrays and shuffle
    imgs_smile = np.array(imgs_smile)
    imgs_nonSmile = np.array(imgs_nonSmile)

    np.random.seed(123)
    idxs = np.random.choice(n, size=n, replace=False)
    imgs_smile = imgs_smile[idxs]
    imgs_nonSmile = imgs_nonSmile[idxs]

    # Train-test split
    n_test = int(n * test_split)
    train_smile = imgs_smile[n_test:]
    train_nonSmile = imgs_nonSmile[n_test:]
    test_smile = imgs_smile[:n_test]
    test_nonSmile = imgs_nonSmile[:n_test]

    return n_test, train_smile, train_nonSmile, test_smile, test_nonSmile
```

## Preprocessing Pipeline Specifications

The preprocessing pipeline employed a series of standardized transformations. All images were resized to  $200 \times 200$  pixel dimensions using TensorFlow's default bilinear interpolation method. Pixel intensity normalization transformed raw pixel values from the integer range  $[0, 255]$  to the floating-point interval  $[0, 1]$  through division by 255. This normalization step facilitates gradient-based optimization and improves numerical stability during model training.

## Dataset Partitioning Strategy

The data partitioning followed a stratified approach maintaining equal class representation in both training and testing

subsets. With the default parameter configuration, the complete dataset contained  $n = 600$  samples per class. A fixed 20% test split allocated  $n_{\text{test}} = \lfloor 600 \times 0.2 \rfloor = 120$  samples per class to the testing set, with the remaining  $n_{\text{train}} = 480$  samples per class designated for training. The shuffle operation employed a fixed random seed `seed = 123` to ensure experimental reproducibility across multiple executions.

## Output Data Structure

The function returns a five-element tuple structured as follows:

```
return = (n_test, train_smile, train_nonSmile,
          test_smile, train_nonSmile)
```

Each image array possesses dimensions  $h \times w \times c$  corresponding to height, width, and color channels. Given the resizing parameters, the resultant arrays maintain shape  $(200, 200, 3)$  for RGB color images.

## Implementation Design Considerations

The implementation emphasizes several critical design principles. Reproducibility is ensured through the fixed random seed initialization. Computational efficiency is achieved through batch processing and vectorized NumPy operations. Standardization of image dimensions and pixel value normalization promotes model convergence across different architectures. Robust error handling manages potential file system inconsistencies, while modular design facilitates parameter tuning for different experimental configurations.

## Integration with Experimental Framework

This preprocessing pipeline generated standardized input data suitable for comparative analysis across three distinct machine learning approaches: the supervised Convolutional Neural Network, the reconstruction-based Autoencoder, and the Random Forest classifier operating on vectorized image pixels. Consistent data preparation across all models ensured fair comparison of generalization capacity, computational efficiency, and robustness metrics as evaluated in the experimental study.

## b. Model Architectures

### 1. Convolutional Neural Network (CNN)

The CNN model was designed with multiple convolutional layers followed by activation functions (ReLU), max-pooling operations, and dense layers that culminate in a sigmoid output for binary classification. This architecture captures spatial hierarchies in the image, making it particularly suitable for expression recognition tasks. The network was trained using binary cross-entropy as the loss function and Adam as the optimization algorithm.

## Data Integration and Model Configuration

The experimental framework integrated preprocessed facial image data into a CNN for binary smile detection. Implementation followed systematic workflow.

## Data Composition and Partitioning

Preprocessing output was transformed into machine learning formats. Image arrays were concatenated into unified feature matrices with binary labels (1=smile, 0=non-smile).

### Listing 2 Data composition and binary label assignment.

```
# Data composition and labels
X_train = np.concatenate([train_smile, train_nonSmile])
y_train = np.array([1]*len(train_smile) + [0]*len(train_nonSmile))

X_test = np.concatenate([test_smile, test_nonSmile])
y_test = np.array([1]*len(test_smile) + [0]*len(test_nonSmile))
```

15% of training data allocated for validation, maintaining class distribution. Result: training (70%), validation (15%), testing (20%).

## CNN Architecture Design

Sequential model with three convolutional blocks (Table 1). Progressive feature abstraction: edges → facial components → smile features.

**TABLE 1: CNN ARCHITECTURE**

| Layer        | Parameters                    |
|--------------|-------------------------------|
| Input        | Shape: (200, 200, 3)          |
| Conv2D       | 32 filters, 3×3 kernel, ReLU  |
| MaxPooling2D | 2×2 pool size                 |
| Conv2D       | 64 filters, 3×3 kernel, ReLU  |
| MaxPooling2D | 2×2 pool size                 |
| Conv2D       | 128 filters, 3×3 kernel, ReLU |
| MaxPooling2D | 2×2 pool size                 |
| Flatten      | -                             |
| Dense        | 128 units, ReLU               |
| Dropout      | Rate: 0.3                     |
| Dense        | 1 unit, Sigmoid               |

Sigmoid output produces  $\hat{y} \in [0, 1]$  probabilities.

## Training Configuration

SGD with momentum:  $\eta = 0.01$ ,  $\beta = 0.9$ . Binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (1)$$

Checkpoint callback preserves best weights. 70 epochs, batch size 50.

### Listing 3 Model compilation and training.

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01,
↳ momentum=0.9)
model.compile(optimizer=optimizer, loss='binary_crossentropy',
↳ metrics=['accuracy'])

checkpoint = ModelCheckpoint(
    filepath='best_model.keras',
    monitor='val_accuracy',
    save_best_only=True,
    mode='max'
)

history = model.fit(
    X_train, y_train,
    epochs=70,
    batch_size=50,
    validation_data=(X_val, y_val),
    callbacks=[checkpoint]
)
```

## Performance Evaluation

Training monitored via accuracy/loss trajectories. Test evaluation thresholds at  $\tau = 0.5$ :

$$\hat{y}_{\text{class}} = \mathbb{I}(\hat{y}_{\text{prob}} > 0.5) \quad (2)$$

Metrics: confusion matrix, recall scores, accuracy.

### Listing 4 Performance evaluation metrics.

```
y_prob = best_model.predict(X_test)
y_pred = (y_prob > 0.5).astype(int).ravel()

cm = confusion_matrix(y_test, y_pred)
recall_per_class = recall_score(y_test, y_pred, average=None)
accuracy = accuracy_score(y_test, y_pred)
```

## Implementation Principles

- Sequential architecture for interpretability
- Stratified partitioning prevents bias
- Checkpointing captures optimal performance
- Momentum SGD balances speed/stability
- Comprehensive evaluation metrics

## 2. Autoencoder

The autoencoder architecture implemented for smile detection operates on a reconstruction-based anomaly detection paradigm. Unlike supervised classifiers, this approach trains exclusively on neutral (non-smiling) facial images, learning to reconstruct them with minimal error. Smiling faces, representing deviations from this learned distribution, produce higher reconstruction errors, enabling binary classification through threshold-based anomaly detection. This unsupervised methodology proves particularly valuable in scenarios where labeled training data is limited or when the goal extends beyond mere classification to include data representation learning and anomaly explanation.

### c. Data Preparation Pipeline

The implementation begins with systematic data extraction from compressed archives containing facial imagery. Label files `SMILE_list.txt` and `NON-SMILE_list.txt` provide filenames for both expression categories, with systematic filename transformations applied to match the `.ppm` image format used throughout the dataset. These transformations ensure consistent file naming conventions despite variations in original label file formats. Images undergo standardized preprocessing: resizing to  $200 \times 200$  pixels, conversion to NumPy arrays, and pixel value normalization to the range  $[0, 1]$  through division by 255. This normalization enhances numerical stability during gradient-based optimization while maintaining relative intensity relationships essential for visual feature recognition.

The complete dataset comprises 600 samples per class, with a fixed 20% test split allocating 120 samples per class to testing, and the remaining 480 samples per class designated for training. Dataset shuffling employs a fixed random seed (seed = 123) to ensure reproducibility across experimental runs while maintaining balanced class representation in all data partitions. This structured partitioning creates distinct training, validation, and testing subsets essential for proper model evaluation and prevention of information leakage between training and assessment phases.

### d. Custom Loss Function Design

A specialized loss function combines binary cross-entropy with weighted L1 reconstruction error, expressed mathematically as  $\mathcal{L}_{\text{combined}} = \alpha \cdot \mathcal{L}_{\text{BCE}} + \beta \cdot \mathcal{L}_{\text{L1-weighted}}$ . The implementation configures  $\alpha = 0.0$  and  $\beta = 1.0$ , emphasizing reconstruction accuracy over probabilistic alignment and effectively utilizing the autoencoder as a pure reconstruction engine rather than a probabilistic model. This design choice reflects the specific requirements of anomaly detection, where precise reconstruction of normal patterns takes precedence over probability calibration.

The weighted L1 component incorporates image brightness information through the formulation  $\mathcal{L}_{\text{L1-weighted}} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \cdot (1 + 2 \cdot \bar{y}_i^2)$ , where  $\bar{y}_i$  represents the mean brightness of the  $i$ -th image. This weighting mechanism prioritizes reconstruction accuracy in brighter facial regions, which typically contain more discriminative features for expression recognition, such as teeth visibility in smiles or eye region illumination changes. By emphasizing reconstruction fidelity in these information-rich areas, the loss function guides the model toward learning features most relevant to expression discrimination while maintaining global reconstruction coherence.

### e. Sobel Edge Enhancement

To enhance feature extraction capabilities, the architecture incorporates Sobel edge detection as a preprocessing step. The implementation defines a custom layer that computes horizontal and vertical gradients for each RGB channel using TensorFlow's built-in Sobel operator. This process generates six additional feature maps—corresponding to horizontal and vertical derivatives for each color channel—which are then concatenated with the original three-channel input.

The resulting nine-channel input tensors provide the model with explicit edge information, enhancing its ability to learn and reconstruct facial contours and expression-relevant features. This edge augmentation proves particularly beneficial for smile detection, where mouth curvature and lip edges serve as primary discriminative features.

The edge extraction function processes input tensors of shape (batch, 200, 200, 3) and returns augmented tensors of shape (batch, 200, 200, 9). This preprocessing occurs within the computational graph as a Lambda layer, ensuring efficient GPU execution and seamless integration with the autoencoder's forward and backward propagation pathways. By providing both raw pixel data and computed edge information, the model receives multi-scale feature representations that facilitate learning of both fine-grained details and broader structural patterns essential for accurate facial expression reconstruction.

### f. Encoder-Decoder Architecture

The encoder implements progressive dimensionality reduction through convolutional downsampling, beginning with input tensors of dimensions  $200 \times 200 \times 9$  that combine RGB values with Sobel edge information. Five convolutional layers with kernel sizes descending from 7 to 3 and filter counts increasing from 9 to 128 systematically extract hierarchical features while reducing spatial dimensions through stride-2 convolutions. Each convolutional operation incorporates batch normalization for stable gradient flow and LeakyReLU activations to mitigate vanishing gradient issues while preserving negative value information potentially useful for edge-based representations. The encoder culminates in a bottleneck layer producing compressed  $6 \times 6 \times 128$  latent representations that encode essential facial characteristics while discarding redundant spatial information.

The decoder symmetrically reconstructs images through transposed convolution equivalents, beginning with the compressed latent representation and progressively restoring spatial dimensions through five upsampling layers. Each decoder stage combines  $2 \times$  spatial upsampling with convolutional filtering using kernel sizes mirroring those in the encoder but with decreasing filter counts from 128 to 8. This symmetrical architecture ensures that the expanding pathway can effectively utilize the hierarchical features encoded in the contracting pathway. The final output layer applies a  $3 \times 3$  convolution with sigmoid activation to produce  $200 \times 200 \times 3$  reconstructions normalized to the  $[0, 1]$  range matching the input preprocessing. Throughout the decoder, LeakyReLU activations maintain gradient flow while batch normalization layers stabilize training dynamics during the complex image generation process.

### g. Training Configuration

Training employs the Adam optimizer with default learning rate ( $\eta = 0.001$ ), balancing rapid convergence with stability across the complex loss landscape of the reconstruction task. Each training iteration processes batches of 32 samples, sufficiently large to provide stable gradient estimates while maintaining memory efficiency for the high-dimensional image data. The model undergoes 100 complete

training epochs, with early stopping implicitly implemented through validation performance monitoring rather than explicit early stopping callbacks. A validation split of 15% of training data provides epoch-wise performance monitoring without compromising the primary training dataset size. Model checkpointing preserves the best-performing weights based on validation loss, ensuring that the final model represents the optimal convergence point rather than potential overfitting in later epochs.

#### ***h. Anomaly Detection Mechanism***

Post-training deployment transforms the autoencoder into an anomaly detector through systematic reconstruction error analysis. The detection pipeline begins with baseline establishment, computing mean reconstruction error  $\mu_{\text{train}}$  and standard deviation  $\sigma_{\text{train}}$  exclusively on non-smile training images. This statistical characterization of normal pattern reconstruction forms the reference distribution against which test samples are evaluated. Threshold definition follows the statistical rule  $\tau = \mu_{\text{train}} + k \cdot \sigma_{\text{train}}$ , with the implementation initially configuring  $k = 1$ , corresponding approximately to the 84th percentile of a normal distribution. This threshold selection balances sensitivity to genuine anomalies with robustness to normal variation in reconstruction quality.

Classification operates on the simple but effective rule that test samples with reconstruction error exceeding  $\tau$  are classified as smiles ( $\hat{y} = 1$ ), while those with errors below the threshold are classified as non-smiles ( $\hat{y} = 0$ ). This binary decision framework transforms the continuous reconstruction error metric into discrete class predictions while preserving the interpretability of the underlying anomaly scores. The threshold parameter  $k$  serves as a tunable sensitivity control, allowing adjustment of the trade-off between false positives and false negatives based on application requirements and error cost asymmetry.

#### ***i. Performance Evaluation Framework***

Evaluation employs multiple complementary metrics assessing both reconstruction quality and classification performance. Reconstruction quality metrics include Mean Absolute Error (MAE) and Mean Squared Error (MSE) computed on the validation set, providing quantitative measures of how accurately the model reproduces input images. Lower values on these metrics indicate better reconstruction fidelity, though optimal classification performance may not strictly correlate with minimal reconstruction error if the model learns to reconstruct both classes equally well. Classification accuracy assessment utilizes confusion matrix analysis comparing predicted versus true labels, with particular attention to class-specific recall rates given the potential imbalance in anomaly detection scenarios.

Error distribution analysis examines quantiles of reconstruction errors across the training set, informing threshold tuning decisions by characterizing the statistical properties of normal pattern reconstruction. This analysis helps determine appropriate  $k$  values for the threshold formula based on desired false positive rates or percentile cutoffs. Visual validation supplements quantitative metrics through side-by-side comparison of original and reconstructed images, provid-

ing intuitive assessment of which facial features the model captures effectively and which reconstruction artifacts might contribute to classification errors. This multi-modal evaluation framework ensures comprehensive assessment of both the autoencoder's reconstruction capabilities and its effectiveness as an anomaly detector for smile classification.

#### ***j. Implementation Advantages***

The implemented autoencoder offers several advantages for smile detection applications. Its unsupervised learning paradigm requires only non-smile images for training, significantly reducing labeling overhead compared to supervised approaches that need balanced labeled datasets. The integration of Sobel edge enhancement provides robust feature extraction capabilities, particularly valuable for detecting expression-relevant facial contours like mouth curvature and lip edges. Adaptive weighting through the brightness-aware loss function prioritizes reconstruction in informative facial regions while maintaining global coherence. The anomaly detection mechanism produces interpretable reconstruction errors that serve as intuitive anomaly scores, offering explanatory value beyond mere classification decisions. Computational efficiency emerges from the model's reduced parameter count compared to equivalent classification CNNs, while still capturing essential facial expression patterns through hierarchical feature learning.

##### ***1. Random Forest***

The Random Forest classifier was employed as a traditional baseline method. Prior to training, images were flattened into one-dimensional vectors, resulting in high-dimensional feature inputs. The classifier constructs a large number of decision trees and aggregates their predictions, leading to robust performance even under simple data representations. Although this method does not leverage spatial correlation, it offers interpretability and low computational cost. The Random Forest classifier implementation employed a distinct methodological approach to facial expression recognition, characterized by feature vectorization and ensemble decision tree learning. This technique operated on flattened image representations, contrasting with the convolutional feature extraction of the CNN and the reconstruction-based methodology of the autoencoder.

##### ***Feature Vectorization Process***

The preprocessing phase transformed the three-dimensional image tensors into one-dimensional feature vectors suitable for traditional machine learning algorithms. This vectorization process reshaped each  $200 \times 200 \times 3$  image into a 120,000-dimensional feature vector, preserving the spatial pixel information while adapting the data structure for Random Forest processing.

**Listing 5** Feature vectorization through image flattening operation.

```
# Image flattening for Random Forest processing
x_train_flat = train_features.reshape(train_features.shape[0], -1)
x_test_flat = test_features.reshape(test_features.shape[0], -1)
```

The transformation operation can be formally expressed as:

$$\phi: \mathbb{R}^{H \times W \times C} \rightarrow \mathbb{R}^{H \cdot W \cdot C} \quad (3)$$

where  $H = 200$ ,  $W = 200$ , and  $C = 3$  represent image height, width, and color channels respectively, resulting in feature vectors of dimension  $d = 200 \times 200 \times 3 = 120,000$ .

**Random Forest Model Specification**

The classification model instantiated a Random Forest classifier with constrained maximum tree depth to prevent overfitting while maintaining computational efficiency. The implementation employed scikit-learn's `RandomForestClassifier` with explicit parameter settings for reproducibility and controlled model complexity.

**TABLE 2:** RANDOM FOREST HYPERPARAMETER CONFIGURATION

| Parameter         | Value            |
|-------------------|------------------|
| max_depth         | 10               |
| random_state      | 1234             |
| n_estimators      | 100 (default)    |
| criterion         | 'gini' (default) |
| max_features      | 'sqrt' (default) |
| min_samples_split | 2 (default)      |

The maximum depth constraint of 10 levels represented a critical regularization mechanism, limiting individual tree complexity while allowing the ensemble to capture meaningful decision boundaries. The fixed random seed ensured deterministic model initialization and consistent experimental reproducibility across multiple executions.

**Training and Prediction Pipeline**

The model training followed the conventional supervised learning paradigm, where the classifier learned decision rules directly from the flattened pixel features. The ensemble approach aggregated predictions from multiple decision trees through majority voting, providing robustness against individual tree biases.

**Listing 6** Random Forest model instantiation, training, and inference.

```
# Model training and prediction
clf = RandomForestClassifier(max_depth=max_depth,
                             random_state=random_state)
clf.fit(x, y)
y_pred = clf.predict(x_test_flat)
```

The prediction mechanism for a given test sample  $\mathbf{x}$  can be expressed as:

$$\hat{y} = \text{mode}(\{h_t(\mathbf{x})\}_{t=1}^T) \quad (4)$$

where  $h_t$  represents the prediction of the  $t$ -th decision tree in the ensemble of size  $T = 100$  (default), and  $\text{mode}(\cdot)$  denotes the majority voting function.

**Comprehensive Performance Evaluation**

The evaluation framework implemented a multi-metric assessment strategy, quantifying classification performance through several complementary measures. This approach provided nuanced insights beyond simple accuracy metrics, particularly valuable for the imbalanced class scenario in facial expression recognition.

**Multi-Metric Assessment Framework**

The evaluation computed four principal performance metrics, each capturing different aspects of classification behavior:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (5)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (6)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (7)$$

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (8)$$

where  $TP$ ,  $TN$ ,  $FP$ , and  $FN$  represent true positives, true negatives, false positives, and false negatives respectively, with "smile" designated as the positive class.

**Listing 7** Comprehensive metric computation for classification evaluation.

```
# Performance metric computation
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, pos_label="smile")
recall = recall_score(y_test, y_pred, pos_label="smile")
f1 = f1_score(y_test, y_pred, pos_label="smile")
cm = confusion_matrix(y_test, y_pred, labels=["smile", "non-smile"])
```

**Results Aggregation and Presentation**

The computed metrics were aggregated into a structured dictionary format, facilitating systematic analysis and comparison across different experimental conditions. The confusion matrix provided detailed insight into class-specific error patterns, while the precision-recall metrics highlighted performance characteristics particularly relevant for the smile detection task.

The label specification `pos_label="smile"` explicitly defined the positive class for binary classification metrics, ensuring consistency in metric interpretation throughout the experimental analysis. This explicit labeling proved essential given the arbitrary numeric encoding (0/1 versus "smile"/"non-smile") variations across different implementation components.

## IV. TRAINING CONFIGURATION

All models were trained under comparable conditions in terms of data splits, epochs, and batch sizes. Training procedures emphasized fairness across approaches to ensure that differences in performance stemmed from the models themselves rather than methodological discrepancies. During training, additional metrics such as loss evolution, prediction accuracy, and validation error were monitored to detect potential overfitting or convergence issues.

## V. RESULTS AND DISCUSSION

In this section, the results of each trial are presented.

### a. CNN Architecture and Results

TABLE 3: CNN LAYER ARCHITECTURE

| Layer                   | Output Shape   | Params         |
|-------------------------|----------------|----------------|
| conv2d                  | (198, 198, 32) | 896            |
| max_pooling2d           | (99, 99, 32)   | 0              |
| conv2d_1                | (97, 97, 64)   | 18496          |
| max_pooling2d_1         | (48, 48, 64)   | 0              |
| conv2d_2                | (46, 46, 128)  | 73856          |
| max_pooling2d_2         | (23, 23, 128)  | 0              |
| flatten                 | (67712)        | 0              |
| dense                   | (128)          | 8667264        |
| dropout                 | (128)          | 0              |
| dense_1                 | (1)            | 129            |
| <b>Total params</b>     |                | <b>8760641</b> |
| <b>Trainable params</b> |                | <b>8760641</b> |

TABLE 4: CNN PERFORMANCE

| Metric             | Value |
|--------------------|-------|
| Recall (Non-smile) | 0.933 |
| Recall (Smile)     | 0.908 |
| Accuracy           | 0.921 |

TABLE 5: CNN PARAMETER DISTRIBUTION

| Layer Category  | Parameters |
|-----------------|------------|
| Convolutional   | 92448      |
| Fully Connected | 8667264    |
| Output Layer    | 129        |
| Total           | 8760641    |

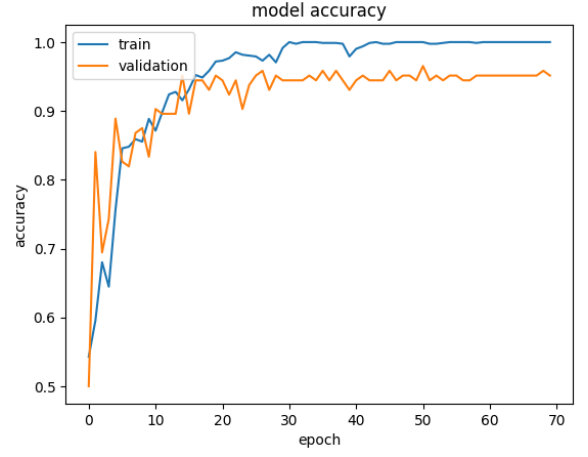


Fig. 2: CNN Training and Validation Accuracy

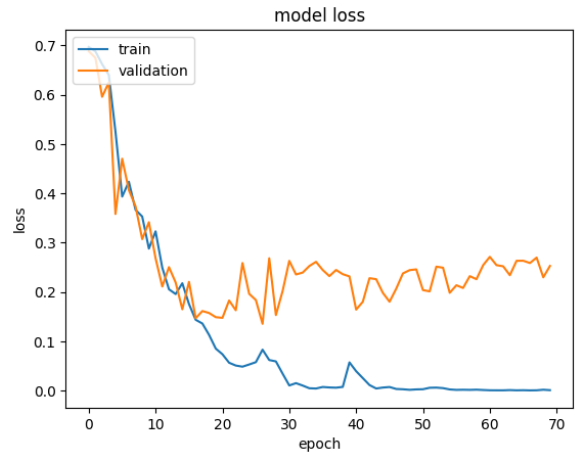


Fig. 3: CNN Training and Validation Loss

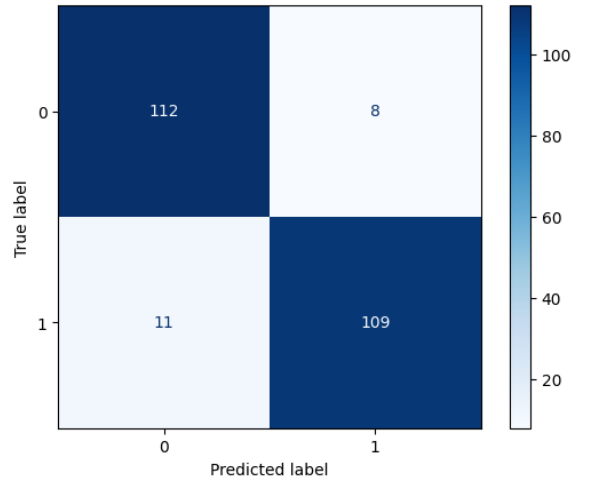


Fig. 4: Confusion Matrix for Classification CNN

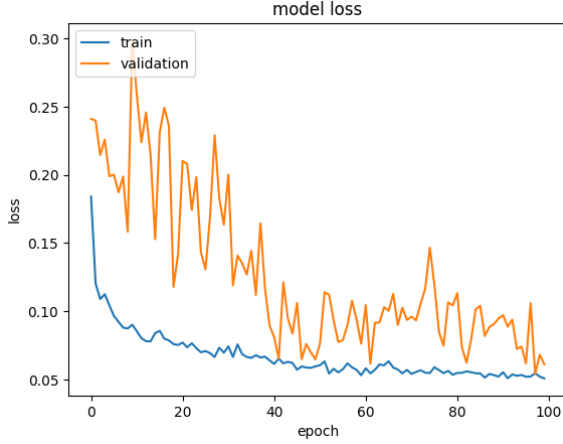
### b. Autoencoder Architecture and Results

TABLE 6: AUTOENCODER CONFUSION MATRIX

|           | Non-smile | Smile |
|-----------|-----------|-------|
| Non-smile | 73        | 47    |
| Smile     | 48        | 72    |

**TABLE 7: AUTOENCODER METRICS**

| Metric    | Value |
|-----------|-------|
| Accuracy  | 0.604 |
| Precision | 0.605 |
| Recall    | 0.600 |
| F1 Score  | 0.603 |

**Fig. 5: Autoencoder Training and Validation Loss****Fig. 6: Autoencoder Model Comparison**

### c. Random Forest

**TABLE 8: RANDOM FOREST CONFUSION MATRIX**

|           | Smile | Non-smile |
|-----------|-------|-----------|
| Smile     | 99    | 21        |
| Non-smile | 10    | 110       |

**TABLE 9: RANDOM FOREST PERFORMANCE**

| Metric    | Value |
|-----------|-------|
| Accuracy  | 0.871 |
| Precision | 0.908 |
| Recall    | 0.825 |
| F1 Score  | 0.865 |

### d. Model Comparison

**TABLE 10: MODEL COMPARISON**

| Model         | Accuracy |
|---------------|----------|
| CNN           | 0.921    |
| Random Forest | 0.871    |
| Autoencoder   | 0.604    |

**TABLE 11: DETAILED AUTOENCODER ARCHITECTURE**

| Layer (Type)            | Output Shape | Params         |
|-------------------------|--------------|----------------|
| Input Layer             | (200,200,3)  | 0              |
| Lambda (Sobel)          | (200,200,6)  | 0              |
| Concatenate             | (200,200,9)  | 0              |
| Conv2D                  | (100,100,9)  | 3,978          |
| LeakyReLU               | (100,100,9)  | 0              |
| Conv2D                  | (50,50,16)   | 3,616          |
| BatchNorm               | (50,50,16)   | 64             |
| LeakyReLU               | (50,50,16)   | 0              |
| Conv2D                  | (25,25,32)   | 4,640          |
| BatchNorm               | (25,25,32)   | 128            |
| LeakyReLU               | (25,25,32)   | 0              |
| Conv2D                  | (13,13,64)   | 18,496         |
| BatchNorm               | (13,13,64)   | 256            |
| LeakyReLU               | (13,13,64)   | 0              |
| Conv2D                  | (7,7,128)    | 73,856         |
| BatchNorm               | (7,7,128)    | 512            |
| LeakyReLU               | (7,7,128)    | 0              |
| <b>Encoder Total</b>    |              | <b>105,446</b> |
| Bottleneck Conv2D       | (7,7,128)    | 147,584        |
| UpSamp + Conv2D         | (14,14,64)   | 73,792         |
| UpSamp + Conv2D         | (28,28,32)   | 18,464         |
| UpSamp + Conv2D         | (52,52,16)   | 4,624          |
| UpSamp + Conv2D         | (100,100,8)  | 1,160          |
| UpSamp + Conv2D         | (200,200,3)  | 219            |
| <b>Decoder Total</b>    |              | <b>98,359</b>  |
| <b>Total Parameters</b> |              | <b>351,389</b> |

## CONCLUSION

This study systematically compared three distinct machine learning approaches—Convolutional Neural Network (CNN), Autoencoder, and Random Forest—for the task of smile detection in facial images. The evaluation, based on accuracy, recall, precision, F1 score, and training dynamics, reveals clear differences in model performance, efficiency, and applicability.

The CNN demonstrated the best overall performance, achieving the highest accuracy (0.921) and strong, balanced recall for both smile (0.908) and non-smile (0.933) classes. Its training and validation accuracy curves show stable convergence with minimal overfitting, indicating strong generalization. Despite its high parameter count ( $\sim 8.76$  million), the CNN effectively captured hierarchical spatial features, making it the most suitable for high-accuracy smile detection in controlled environments where computational resources are not a primary constraint.

The Random Forest classifier delivered robust performance as a traditional baseline, with an accuracy of 0.871, high precision (0.908), and a solid F1 score (0.865). Its recall for the smile class was 0.825, which is respectable though lower than the CNN's, while recall for non-smile was implicitly high given the overall accuracy and confusion matrix. With significantly lower computational complexity and inherent interpretability, Random Forest serves as a practical and efficient alternative, especially when training data is limited, hardware resources are constrained, or model transparency is required.

The Autoencoder, operating on an unsupervised anomaly

detection paradigm, achieved moderate accuracy (0.604) with a recall of 0.600, reflecting its specialized design rather than direct classification optimization. Its confusion matrix indicates difficulty in cleanly distinguishing smiles from non-smiles, likely due to overlapping reconstruction errors between classes. The training loss curves show steady decrease but plateau at a higher error level, suggesting limited discriminative power for this specific binary task. However, the Autoencoder's ability to operate with only non-smile training data and its interpretable reconstruction errors provide unique value for scenarios where labeled data is scarce, anomaly explanation is needed, or the learning objective extends beyond mere classification to include representation learning.

From a practical standpoint, the choice of model should be guided by specific application requirements. For high-stakes applications such as affective computing or security systems, the CNN is recommended due to its superior accuracy and balanced recall across classes. For resource-limited deployments or scenarios demanding model interpretability, Random Forest offers a compelling balance of performance, efficiency, and transparency. In unsupervised or semi-supervised settings where labeling is expensive or impractical, the Autoencoder provides a viable, though less accurate, alternative with moderate recall and the advantage of learning from normal patterns alone.

Future work may extend this comparison by incorporating strategies to enhance model robustness and performance. Potential directions include improving Autoencoder performance through hybrid approaches that combine reconstruction error with learned discriminative features, applying transfer learning or advanced data augmentation to boost generalization across all models, and integrating facial landmark detection to reduce reliance on raw pixel data and improve feature invariance across diverse facial structures and imaging conditions.

In summary, while CNNs excel in accuracy and balanced recall for smile detection, each model presents distinct trade-offs. The optimal selection depends on a careful consideration of performance needs, computational resources, data availability, and the desired level of model interpretability within the target application context.



**Fig. 7:** Face Comparison

## VI. GIT REPOSITORY

JuanPabloDP/repository